

```
# -*- coding: utf-8 -*-
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms
from torch.utils.data import DataLoader, Dataset
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image, ImageDraw, ImageFont
import random
import os
import torch.nn.functional as F
import torchvision.transforms.functional as TF
import string
import math
```

```
!wget -nc -O times.ttf https://raw.githubusercontent.com/jx06T/audio-on-slides/refs/heads/main/times.ttf
# !wget -nc -O EB-Garamond.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/EB-Garamond.ttf

# !wget -nc -O PlaywriteAUTAS.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/PlaywriteAUTAS.ttf
# !wget -nc -O DancingScript.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/DancingScript.ttf
# !wget -nc -O PlaywriteDESAS.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/PlaywriteDESAS.ttf

# !wget -nc -O Yozai.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/Yozai.ttf
!wget -nc -O YuPearl.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/YuPearl-Light.ttf

# !wget -nc -O NotoSerifTC.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/NotoSerifTC.ttf
# !wget -nc -O ChironGoRoundTC.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/ChironGoRoundTC.ttf
# !wget -nc -O NotoSansHK.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/NotoSansHK.ttf

# !wget -nc -O kaiu.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/kaiu.ttf
!wget -nc -O tegaki_zatsu.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/tegaki_zatsu.ttf
!wget -nc -O GenJyuuGothic-Light.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/GenJyuuGothic-Light.ttf
!wget -nc -O NotoSansTC-Regular.ttf https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/NotoSansTC-Regular.ttf
```

```
--2026-04-15 14:27:12-- https://raw.githubusercontent.com/jx06T/audio-on-slides/refs/heads/main/times.ttf
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1206884 (1.2M) [application/octet-stream]
Saving to: 'times.ttf'
```

```
times.ttf      100%[=====>] 1.15M --.-KB/s in 0.006s
```

```
2026-04-15 14:27:13 (191 MB/s) - 'times.ttf' saved [1206884/1206884]
```

```
--2026-04-15 14:27:13-- https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/YuPearl-Light.ttf
Resolving github.com (github.com)... 20.205.243.166
Connecting to github.com (github.com)|20.205.243.166|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/jx06T/audio-on-slides/refs/heads/main/YuPearl-Light.ttf [following]
--2026-04-15 14:27:14-- https://raw.githubusercontent.com/jx06T/audio-on-slides/refs/heads/main/YuPearl-Light.ttf
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5471636 (5.2M) [application/octet-stream]
Saving to: 'YuPearl.ttf'
```

```
YuPearl.ttf    100%[=====>] 5.22M --.-KB/s in 0.07s
```

```
2026-04-15 14:27:16 (78.6 MB/s) - 'YuPearl.ttf' saved [5471636/5471636]
```

```
--2026-04-15 14:27:16-- https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/tegaki_zatsu.ttf
Resolving github.com (github.com)... 20.205.243.166
Connecting to github.com (github.com)|20.205.243.166|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/jx06T/audio-on-slides/refs/heads/main/tegaki_zatsu.ttf [following]
--2026-04-15 14:27:16-- https://raw.githubusercontent.com/jx06T/audio-on-slides/refs/heads/main/tegaki_zatsu.ttf
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 28588168 (27M) [application/octet-stream]
Saving to: 'tegaki_zatsu.ttf'
```

```
tegaki_zatsu.ttf 100%[=====>] 27.26M --.-KB/s in 0.1s
```

```
2026-04-15 14:27:19 (193 MB/s) - 'tegaki_zatsu.ttf' saved [28588168/28588168]
```

```
--2026-04-15 14:27:19-- https://github.com/jx06T/audio-on-slides/raw/refs/heads/main/GenJyuuGothic-Light.ttf
Resolving github.com (github.com)... 20.205.243.166
Connecting to github.com (github.com)|20.205.243.166|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/jx06T/audio-on-slides/refs/heads/main/GenJyuuGothic-Light.ttf [following]
```

```
--2026-04-15 14:27:19-- https://raw.githubusercontent.com/ix06T/audio-on-slides/refs/heads/main/GenJyuuGothic-Light.ttf
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10387336 (9.9M) [application/octet-stream]
Saving to: 'GenJyuuGothic-Light.ttf'
```

GenJyuuGothic-Light 100%[=====>] 9.91M --KB/s in 0.04s

2026-04-15 14:27:20 (232 MB/s) - 'GenJyuuGothic-Light.ttf' saved [10387336/10387336]

```
# 設置設備
# 移動 大大
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class Config:
    def __init__(self):
        # 實驗核心變因
        self.bottleneck_size = 8          # n size (1, 2, 4, 8, 16...)
        self.kernel_size = 4              # 卷積核大小 (舊版是 6)
        self.padding = 1                  # 填充大小 (配合 k=6, s=2 需要 p=2 才能剛好除以 2)
        self.fixed_layers = 5

        self.lambda_l1 = 10              # L1 Loss 權重 (L1 與 D-loss 的比例)

        self.aug_scale = (0.7, 1.35)     # 增強縮放範圍
        self.aug_translate = (0.6, 0.6)  # 增強平移範圍
        self.aug_mask_prob = 0           # 遮罩機率
        self.aug_degrees = 0

        # GAN 基礎參數
        self.nz = 256                    # 噪聲維度
        self.ngf = 64                    # 生成器基礎特徵圖數
        self.ndf = 64                    # 判別器基礎特徵圖數
        self.image_size = 64             # 圖像大小
        self.nc = 1                      # 通道數
        self.lambda_gp = 10              # WGAN-GP 梯度懲罰
        self.lr = 0.0004
        self.batch_size = 32
        self.betas = (0.0, 0.9)
        self.num_epochs = 600           # 示範用，實際可設多一點

        # 訓練控制
        self.n_critic = 2               # 判別器訓練次數 / 生成器訓練次數

        # self.missing_chars = ["R", "G", "E", "Z", "A", "3", "7"]
        self.missing_chars = ["e", "w", "p", "m", "t", "R", "G", "E", "Z", "I", "D", "M", "b", "u", "U", "3", "5", "7"]
        self.missing_chars = []

    def __str__(self):
        return str(self.__dict__)

# 初始化配置
cfg = Config()
print(f"目前配置: n={cfg.bottleneck_size}, 固定層數={cfg.fixed_layers}")
print(f"增強設定: Rot={cfg.aug_degrees}, Trans={cfg.aug_translate}, Scale={cfg.aug_scale}")
```

目前配置: n=8, 固定層數=5

增強設定: Rot=0, Trans=(0.6, 0.6), Scale=(0.7, 1.35)

```
class PairedFontDataset(Dataset):
    def __init__(self, font_path_a, font_path_b, config, num_samples=5000, invert=True):
        self.num_samples = num_samples
        self.img_size = config.image_size
        self.invert = invert
        self.cfg = config

        # 字體設置
        try:
            self.font_a = ImageFont.truetype(font_path_a, size=int(config.image_size * 1.1))
            self.font_b = ImageFont.truetype(font_path_b, size=int(config.image_size * 1.06))
        except IOError:
            raise RuntimeError("字體文件未找到，請檢查路徑。")

        # --- 1. 定義全集 (數字 + 大寫 + 小寫) ---
        digits = [str(i) for i in range(10)]
        upper = list(string.ascii_uppercase)
        lower = list(string.ascii_lowercase)
        char_string = list(
            # "一丨丿乙丨二亅人亻儿入八冂冫冫 几凵刀リ力勾匕匕十卜尸厶冂又口土夂夕大女子宀寸小尢兀尸中山叺工己巾干幺广乚升弋"
            "一丨丿乙丨二亅人亻儿入八冂冫冫 几凵刀リ力勾匕匕十卜尸厶冂又口土夂夕大女子宀寸小尢兀尸中山叺工己巾干幺广乚升弋"
            "一丨丿乙丨日月木水火土田目手心人入八山石竹米艸衣言金雨風魚馬鳥女子口足卅彳彳 小イ 小大中上下左右天地雲川星耳鼻頭身男"
        )
        all_candidates = digits + upper + lower + char_string
```

```

# --- 2. 硬排除 (Hard Exclude) ---
# 這些是「渲染效果很差」或「過寬」的字，完全不使用
hard_exclude = {}

valid_chars = [ch for ch in all_candidates if ch not in hard_exclude]

# --- 3. 根據 Config 分組 ---
# Missing Set: 雖然是有效字，但在訓練時假裝沒有 Target (Mask=0)
self.missing_set = [c for c in valid_chars if c in config.missing_chars]

# Common Set: 有效且有 Target 的普通訓練字 (Mask=1)
self.characters = [c for c in valid_chars if c not in self.missing_set]

print(f"總有效字符: {len(valid_chars)}")
print(f"└── 普通訓練字符 (Common): {len(self.characters)} 個")
print(f"└── 缺失測試字符 (Missing/Zero-shot): {len(self.missing_set)}")

def _render_char_to_tensor(self, char_str: str, font: ImageFont) -> torch.Tensor:
    # 畫布稍大一點以免邊緣裁切
    canvas_size = (int(self.img_size * 2), int(self.img_size * 2))
    img_pil = Image.new("L", canvas_size, color=255)
    draw = ImageDraw.Draw(img_pil)

    bbox = draw.textbbox((0, 0), char_str, font=font)
    text_width = bbox[2] - bbox[0]
    text_height = bbox[3] - bbox[1]
    position = ((canvas_size[0] - text_width) / 2, (canvas_size[1] - text_height) / 2)
    draw.text(position, char_str, font=font, fill=0)

    img_tensor = transforms.ToTensor()(img_pil)

    # 裁剪內容邏輯 (保持原樣或微調)
    non_white = torch.where(img_tensor < 1.0)
    if non_white[0].numel() == 0: return torch.ones(1, self.img_size, self.img_size)

    top, bottom = torch.min(non_white[1]), torch.max(non_white[1])
    left, right = torch.min(non_white[2]), torch.max(non_white[2])

    # 增加一點 padding
    pad = 5
    img_cropped = img_tensor[:, max(0, top-pad):min(canvas_size[1], bottom+pad),
                                                                    max(0, left-pad):min(canvas_size[0], right+pad)]

    # Resize and Center on Final Canvas
    final_canvas = torch.ones(1, self.img_size, self.img_size)

    # 保持長寬比縮放
    c_h, c_w = img_cropped.shape[1], img_cropped.shape[2]
    ratio = min((self.img_size * 0.8) / c_h, (self.img_size * 0.8) / c_w)
    new_h, new_w = int(c_h * ratio), int(c_w * ratio)

    resized_img = F.interpolate(img_cropped.unsqueeze(0), size=(new_h, new_w), mode='bilinear', align_corners=False).

    y_off = (self.img_size - new_h) // 2
    x_off = (self.img_size - new_w) // 2
    final_canvas[:, y_off:y_off+new_h, x_off:x_off+new_w] = resized_img

    return final_canvas

def __len__(self):
    return self.num_samples

def __getitem__(self, idx):
    char_str = random.choice(self.characters)
    label = 0 # 簡化，本任務不依賴標籤分類

    img_a = 1.0 - self._render_char_to_tensor(char_str, self.font_a)
    img_b = 1.0 - self._render_char_to_tensor(char_str, self.font_b)

    # 同步增強
    params = transforms.RandomAffine.get_params(
        degrees=(-self.cfg.aug_degrees, self.cfg.aug_degrees), translate=self.cfg.aug_translate,
        scale_ranges=self.cfg.aug_scale, shears=None,
        img_size=[self.img_size, self.img_size]
    )

    aug_a = TF.affine(img_a, *params, interpolation=transforms.InterpolationMode.BILINEAR, fill=0)
    aug_b = TF.affine(img_b, *params, interpolation=transforms.InterpolationMode.BILINEAR, fill=0)

    # 隨機遮罩

```

```

if random.random() < self.cfg.aug_mask_prob:
    # 簡單實作: 隨機遮蔽一塊
    mask_size = int(self.img_size * 0.4)
    mx = random.randint(0, self.img_size - mask_size)
    my = random.randint(0, self.img_size - mask_size)
    aug_a[:, my:my+mask_size, mx:mx+mask_size] = 0
    aug_b[:, my:my+mask_size, mx:mx+mask_size] = 0

if not self.invert:
    return 1.0 - aug_a, 1.0 - aug_b, label
return aug_a, aug_b, label

```

```

class DynamicGenerator(nn.Module):
    def __init__(self, config):
        super(DynamicGenerator, self).__init__()
        self.nz = config.nz
        self.bottleneck_size = config.bottleneck_size

        # 計算需要縮小幾次 (Stride=2)
        needed_downsamples = int(math.log2(config.image_size / config.bottleneck_size))

        # --- 1. Content Encoder (source_encoder) ---
        layers = []
        in_ch, out_ch = config.nc, config.ngf

        for i in range(config.fixed_layers):
            is_last = (i == config.fixed_layers - 1)

            # 優先進行下採樣, 次數夠了就改用 stride=1 保持特徵圖大小
            if i < needed_downsamples:
                # 下採樣層 (使用 cfg 的 k=6, p=2)
                k, s, p = config.kernel_size, 2, config.padding
            else:
                # 保持層 (使用標準 ResNet 設定 k=3, s=1, p=1 以維持穩定)
                k, s, p = 3, 1, 1

            layers.append(nn.Conv2d(in_ch, out_ch, k, s, p, bias=False))
            if not is_last:
                layers.append(nn.BatchNorm2d(out_ch))
                layers.append(nn.LeakyReLU(0.2, inplace=True))

            in_ch = out_ch
            if i < needed_downsamples: out_ch = min(out_ch * 2, 512)

        self.source_encoder = nn.Sequential(*layers) # <--- 命名回歸
        self.enc_out_ch = in_ch

        # --- 2. Decoder (main_branch) ---
        layers = []
        dec_in_ch = self.enc_out_ch + config.nz
        num_flat_layers = config.fixed_layers - needed_downsamples

        for i in range(config.fixed_layers):
            is_last = (i == config.fixed_layers - 1)
            is_flat = i < num_flat_layers # 先做保持層, 最後才上採樣

            if is_flat:
                k, s, p = 3, 1, 1
                dec_out_ch = dec_in_ch
            else:
                k, s, p = config.kernel_size, 2, config.padding
                dec_out_ch = dec_in_ch // 2

            if is_last: dec_out_ch = config.nc

            if is_flat:
                layers.append(nn.Conv2d(dec_in_ch, dec_out_ch, k, s, p, bias=False))
            else:
                layers.append(nn.ConvTranspose2d(dec_in_ch, dec_out_ch, k, s, p, bias=False))

            if not is_last:
                layers.append(nn.BatchNorm2d(dec_out_ch)); layers.append(nn.ReLU(True))
                dec_in_ch = dec_out_ch
            else:
                layers.append(nn.Sigmoid())

        self.main_branch = nn.Sequential(*layers) # <--- 命名回歸

    def forward(self, noise, src):
        # 1. 提取特徵
        feat = self.source_encoder(src)

```

```
# 2. 處理噪聲
# if noise.dim() == 2: noise = noise.view(noise.size(0), noise.size(1), 1, 1)
# noise_exp = noise.expand(-1, -1, self.bottleneck_size, self.bottleneck_size)
noise_exp = noise
# 3. 合併
combined = torch.cat((feat, noise_exp), dim=1)

# 4. 生成 (確保不用再手動 interpolate)
return self.main_branch(combined)
```

```
class Discriminator(nn.Module):
    def __init__(self, config):
        super(Discriminator, self).__init__()
        nc = config.nc
        ndf = config.ndf

        self.main = nn.Sequential(
            # 64 -> 32
            nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # 32 -> 16
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # 16 -> 8
            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # 8 -> 4
            nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # Output decision: 4 -> 1
            nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
        )

    def forward(self, input):
        return self.main(input).view(-1)
```

```
def compute_gradient_penalty(D, real_samples, fake_samples, device):
    alpha = torch.rand(real_samples.size(0), 1, 1, 1, device=device)
    interpolates = (alpha * real_samples + (1 - alpha) * fake_samples).requires_grad_(True)
    d_interpolates = D(interpolates)
    fake = torch.ones(real_samples.size(0), device=device)
    gradients = torch.autograd.grad(
        outputs=d_interpolates, inputs=interpolates, grad_outputs=fake,
        create_graph=True, retain_graph=True, only_inputs=True
    )[0]
    gradients = gradients.view(gradients.size(0), -1)
    gradient_penalty = ((gradients.norm(2, dim=1) - 1) ** 2).mean()
    return gradient_penalty
```

```
def plot_gan_results(source_imgs, target_imgs, fake_imgs, labels, epoch, config, title_suffix):
    """
    繪製 GAN 生成結果 (適配 Config 物件)
    """
    # 確保數據在 CPU 上並轉為 numpy, 且去掉多餘維度
    source_imgs = source_imgs.detach().cpu()
    target_imgs = target_imgs.detach().cpu()
    fake_imgs = fake_imgs.detach().cpu()

    n_samples = source_imgs.size(0)
    # 防呆: 如果只有一張圖
    if n_samples == 1:
        fig, axes = plt.subplots(1, 3, figsize=(9, 3))
        axes = axes[np.newaxis, :] # 增加維度以便下面統一處理
        # 這裡需要手動調整一下結構, 但為了通用性, 建議測試時至少 batch=2
    else:
        fig, axes = plt.subplots(3, n_samples, figsize=(2 * n_samples, 6))

    main_title = (f"{title_suffix} | Epoch {epoch}/{config.num_epochs}\n"
                  f"n={config.bottleneck_size}, L1_w={config.lambda_l1}\n"
                  f"lr={config.lr}, bs={config.batch_size}")

    for j in range(n_samples):
        # 處理 labels (如果是 Tensor 就轉文字, 如果是 list 就直接用)
        lbl = labels[j].item() if isinstance(labels, torch.Tensor) else labels[j]

        # 來源圖像
        ax_src = axes[0, j] if n_samples > 1 else axes[0]
        ax_src.imshow(source_imgs[j].squeeze(), cmap='gray')
        ax_src.set_title(f"Source ({lbl})", fontsize=8)
        ax_src.axis('off')
```

```

# 目標圖像
ax_tgt = axes[1, j] if n_samples > 1 else axes[1]
ax_tgt.imshow(target_imgs[j].squeeze(), cmap='gray')
ax_tgt.set_title(f"Target ({lbl})", fontsize=8)
ax_tgt.axis('off')

# 生成圖像
ax_gen = axes[2, j] if n_samples > 1 else axes[2]
ax_gen.imshow(fake_imgs[j].squeeze(), cmap='gray')
ax_gen.set_title(f"Generated", fontsize=8)
ax_gen.axis('off')

plt.suptitle(main_title)
plt.tight_layout(rect=[0, 0.03, 1, 0.9])
plt.show()

```

```

def plot_training_losses(history, config):
    plt.figure(figsize=(10, 5))

    if 'D_loss' in history:
        plt.plot(history['D_loss'], label='Discriminator Loss', alpha=0.6)

    if 'G_loss' in history:
        plt.plot(history['G_loss'], label='Generator Loss (Total)', alpha=0.6)

    # 如果有記錄 L1 Loss, 也可以畫出來 (通常數值較大, 建議用雙軸或分開畫, 這裡先畫在一起)
    if 'L1_loss' in history:
        plt.plot(history['L1_loss'], label='L1 Pixel Loss', linestyle='--', alpha=0.5)

    # 繪製 D 和 G 的 Loss
    if 'loss_G_adv' in history:
        plt.plot(history['loss_G_adv'], label='G adv Loss', linestyle='--', alpha=0.2)

    plt.xlabel('Iterations (logged step)')
    plt.ylabel('Loss')
    plt.legend()
    plt.title(f'Training Losses (n={config.bottleneck_size}, L1 ratio={config.lambda_l1})')
    plt.grid(True, alpha=0.3)
    plt.show()

```

```

def preprocess_char(dataset, char_str):
    """
    統一的前處理邏輯: 確保測試時的輸入跟訓練時完全一樣 (包含反色邏輯)
    """
    # 1. 取得基礎渲染圖 (0=字, 1=背景)
    # 注意: 這裡假設 _render_char_to_tensor 回傳的是 PIL 預設 (白底黑字 render 出來 0=黑色字)
    rendered = dataset._render_char_to_tensor(char_str, dataset.font_a)

    # 2. 模擬 Dataset.__getitem__ 的邏輯
    # 在 Dataset 中, 第一步是 img_a = 1.0 - img_a_base (變成 1=字, 0=背景)
    img_tensor = 1.0 - rendered

    # 3. 處理反色配置
    if not dataset.invert:
        # 如果設定為白底黑字 (invert=False), 需要再反轉一次回來
        # 變成 0=字 (黑), 1=背景 (白)
        img_tensor = 1.0 - img_tensor

    # 如果 dataset.invert=True, 則保持步驟 2 的結果 (1=字, 0=背景)

    return img_tensor

def test_specific_chars(G, dataset, config, epoch, chars_to_test=None):
    if chars_to_test is None:
        # chars_to_test = ["a","t","R", "G", "E", "Z", "A", "3","B","H","U", "7"]
        chars_to_test = ["t","R", "G", "7","田","名","朝","明"]

    # G.eval() # 切換到評估模式 (影響 BatchNorm)
    source_list = []
    target_list = []

    # 使用與訓練一致的處理邏輯
    for char in chars_to_test:
        # 處理來源圖 (Input)
        img_a = preprocess_char(dataset, char)
        source_list.append(img_a)

```

```

# 處理目標圖 (Target) - 僅供畫圖參考用
# 注意: 我們通常想看目標圖的樣子, 所以也要用同樣的字元去 render font_b
# 但這裡其實 font_b 的處理邏輯要參考 font_a, 通常是一樣的
# 我們直接借用 preprocess_char, 只是要把內部的 font 改成 font_b
# 為了方便, 我們手動做一次 font_b 的渲染
raw_b = dataset._render_char_to_tensor(char, dataset.font_b)
img_b = 1.0 - raw_b
if not dataset.invert:
    img_b = 1.0 - img_b
target_list.append(img_b)

# 堆疊成 Batch 並送入 GPU
src_batch = torch.stack(source_list).to(device)
tgt_batch = torch.stack(target_list).to(device)

# 生成
with torch.no_grad():
    z = torch.randn(len(chars_to_test), config.nz, config.bottleneck_size, config.bottleneck_size).to(device)
    fake_batch = G(z, src_batch)

# 呼叫繪圖函數
plot_gan_results(src_batch, tgt_batch, fake_batch, chars_to_test,
                  epoch, config, "Specific Test Characters")

G.train() # 記得切回訓練模式

```

```

def generate_specific_text(model, dataset, config, text_input, mode='detail'):
    """
    針對特定字串進行生成, 並顯示 Input (Source), Target (Real), Generated (Fake)。

    Args:
        model: 訓練好的生成器 (G)
        dataset: 用於渲染來源字體 (Times) 的 dataset 物件
        config: 設定檔
        text_input (str): 想生成的文字, 例如 "Hello"
        mode (str):
            'detail': 顯示每個字的獨立對比圖 (3行: Source, Target, Gen)
            'sentence': 拼成整行句子 (3行: Source句子, Target句子, Gen句子)
    """
    # model.eval() # 視情況開啟, 有些模型 train mode 生成效果反而較好

    source_tensors = []
    target_tensors = [] # 新增: 用來存目標圖

    # 1. 準備圖像
    print(f"正在生成文字: {text_input}")
    for char in text_input:
        # A. 渲染來源 (Times New Roman)
        rendered_a = dataset._render_char_to_tensor(char, dataset.font_a)
        # B. 渲染目標 (Handwritten / Garamond)
        rendered_b = dataset._render_char_to_tensor(char, dataset.font_b)

        # 處理顏色反轉
        if dataset.invert:
            rendered_a = 1.0 - rendered_a
            rendered_b = 1.0 - rendered_b

        source_tensors.append(rendered_a)
        target_tensors.append(rendered_b)

    # 堆疊成 Batch
    src_batch = torch.stack(source_tensors).to(device)
    tgt_batch = torch.stack(target_tensors) # Target 只需要在 CPU 等著畫圖

    bs = len(text_input)

    # 2. 生成
    with torch.no_grad():
        # 使用 4x4 的噪聲 (根據我們先前的討論, 這樣效果最好)
        z = torch.randn(bs, config.nz, config.bottleneck_size, config.bottleneck_size).to(device)
        fake_imgs = model(z, src_batch)

    # 轉回 CPU 準備繪圖
    src_batch = src_batch.cpu()
    fake_imgs = fake_imgs.cpu()

    # 3. 視覺化
    if mode == 'detail':
        # 修改: 變成 3 行 (Source, Target, Generated)
        fig, axes = plt.subplots(3, bs, figsize=(bs * 2, 6))

```

```

# 防呆: 如果只有一個字, axes 是一維陣列, 需要增加維度方便索引
if bs == 1:
    axes = axes[:, np.newaxis]

for i in range(bs):
    # 第一行: 來源 (Input)
    axes[0, i].imshow(src_batch[i].squeeze(), cmap='gray')
    axes[0, i].set_title(f"Input: {text_input[i]}")
    axes[0, i].axis('off')

    # 第二行: 目標 (Target / Real)
    axes[1, i].imshow(tgt_batch[i].squeeze(), cmap='gray')
    axes[1, i].set_title("Target (Real)")
    axes[1, i].axis('off')

    # 第三行: 生成 (Generated / Fake)
    axes[2, i].imshow(fake_imgs[i].squeeze(), cmap='gray')
    axes[2, i].set_title("Generated")
    axes[2, i].axis('off')

plt.tight_layout()
plt.show()

elif mode == 'sentence':
    # 將生成的圖水平拼接成一行字
    src_list = [img.squeeze().numpy() for img in src_batch]
    tgt_list = [img.squeeze().numpy() for img in tgt_batch]
    gen_list = [img.squeeze().numpy() for img in fake_imgs]

    # 水平拼接 (拼成句子)
    src_sentence = np.concatenate(src_list, axis=1)
    tgt_sentence = np.concatenate(tgt_list, axis=1)
    gen_sentence = np.concatenate(gen_list, axis=1)

    # 垂直拼接 (將三個句子疊在一起比較)
    # 為了美觀, 可以在句子之間加一點空白分隔線
    spacer = np.ones((10, src_sentence.shape[1])) # 10 pixel 高的白線
    if dataset.invert: spacer = spacer * 0 # 如果是黑底白字, 分隔線用黑的

    full_comparison = np.concatenate([
        src_sentence,
        spacer,
        tgt_sentence,
        spacer,
        gen_sentence
    ], axis=0)

    plt.figure(figsize=(15, 6)) # 加大高度
    plt.imshow(full_comparison, cmap='gray')
    plt.title(f"Sentence Comparison: Top=Input, Mid=Target, Bot=Generated")
    plt.axis('off')
    plt.show()

```

```

def preview_dataset_samples(dataset, n=6):
    """
    預覽數據集: 顯示 Source, Target 以及兩者的疊加圖 (Overlay)
    Overlay 有助於檢查幾何增強是否同步應用在成對圖像上。
    """
    # 建立 3 行 n 列的圖表 (Source, Target, Overlay)
    fig, axes = plt.subplots(3, n, figsize=(n * 2.5, 6))

    # 隨機抽樣
    indices = [random.randint(0, len(dataset)-1) for _ in range(n)]

    for i, idx in enumerate(indices):
        # 取得資料
        item = dataset[idx]
        if isinstance(item, tuple) and len(item) == 3:
            src, tgt, label = item
        else:
            # 防呆
            src, tgt = item[0], item[1]
            label = ""

        # 轉為 Numpy
        src_np = src.squeeze().numpy()
        tgt_np = tgt.squeeze().numpy()

        # 1. 第一行: Source (Times)
        axes[0, i].imshow(src_np, cmap='gray')
        axes[0, i].set_title(f"Source", fontsize=9)
        axes[0, i].axis('off')

```



```

# 2. 第二行: Target (Garamond/Handwritten)
axes[1, i].imshow(tgt_np, cmap='gray')
axes[1, i].set_title(f"Target", fontsize=9)
axes[1, i].axis('off')

# 3. 第三行: Overlay (疊加比較)
# 底層用 Target (灰色), 上層用 Source (紅色半透明)
# 這樣可以清楚看到結構對齊的情況
axes[2, i].imshow(tgt_np, cmap='gray', alpha=1.0)
axes[2, i].imshow(src_np, cmap='Reds', alpha=0.5) # 使用紅色 colormap 區分
axes[2, i].set_title(f"Overlay", fontsize=9)
axes[2, i].axis('off')

```

```

plt.suptitle("Dataset Preview: Source (Top), Target (Mid), Overlay (Bottom)")
plt.tight_layout()
plt.show()

```

```

def visualize_results(G, dataset, config, title="Result"):
    # G.eval()
    src_cpu, tgt_cpu, _ = dataset[0] # 取一組樣本
    # 增加 batch 維度
    src = src_cpu.unsqueeze(0).to(device)
    z = torch.randn(1, config.nz, config.bottleneck_size, config.bottleneck_size).to(device)

    with torch.no_grad():
        fake = G(z, src)

    fig, ax = plt.subplots(1, 3, figsize=(10, 4))
    ax[0].imshow(src_cpu.squeeze(), cmap='gray'); ax[0].set_title("Input (Times)")
    ax[1].imshow(fake.cpu().squeeze(), cmap='gray'); ax[1].set_title(f"Generated (n={config.bottleneck_size})")
    ax[2].imshow(tgt_cpu.squeeze(), cmap='gray'); ax[2].set_title("Target (Garamond)")
    plt.suptitle(title)
    plt.show()

```

```

# 1. 安裝
!pip install torchinfo

from torchinfo import summary

# 2. 建立模型 (假設 n=4)
model_test = DynamicGenerator(cfg).to(device)

# 3. 顯示架構
# input_data 是一個 list, 分別對應 forward 的兩個參數: (noise, source_imgs)
print(f"=== Model Summary (n={cfg.bottleneck_size}) ===")
summary(model_test, input_data=[
    torch.randn(1, cfg.nz, cfg.bottleneck_size, cfg.bottleneck_size).to(device),
    torch.randn(1, cfg.nc, cfg.image_size, cfg.image_size).to(device) # Source Image
])

```

```

Collecting torchinfo
  Downloading torchinfo-1.8.0-py3-none-any.whl.metadata (21 kB)
Downloading torchinfo-1.8.0-py3-none-any.whl (23 kB)
Installing collected packages: torchinfo
Successfully installed torchinfo-1.8.0
=== Model Summary (n=8) ===

```

Layer (type:depth-idx)	Output Shape	Param #
DynamicGenerator	[1, 1, 64, 64]	--
├─Sequential: 1-1	[1, 512, 8, 8]	--
│ └─Conv2d: 2-1	[1, 64, 32, 32]	1,024
│ └─BatchNorm2d: 2-2	[1, 64, 32, 32]	128
│ └─LeakyReLU: 2-3	[1, 64, 32, 32]	--
│ └─Conv2d: 2-4	[1, 128, 16, 16]	131,072
│ └─BatchNorm2d: 2-5	[1, 128, 16, 16]	256
│ └─LeakyReLU: 2-6	[1, 128, 16, 16]	--
│ └─Conv2d: 2-7	[1, 256, 8, 8]	524,288
│ └─BatchNorm2d: 2-8	[1, 256, 8, 8]	512
│ └─LeakyReLU: 2-9	[1, 256, 8, 8]	--
│ └─Conv2d: 2-10	[1, 512, 8, 8]	1,179,648
│ └─BatchNorm2d: 2-11	[1, 512, 8, 8]	1,024
│ └─LeakyReLU: 2-12	[1, 512, 8, 8]	--
│ └─Conv2d: 2-13	[1, 512, 8, 8]	2,359,296
├─Sequential: 1-2	[1, 1, 64, 64]	--
│ └─Conv2d: 2-14	[1, 768, 8, 8]	5,308,416
│ └─BatchNorm2d: 2-15	[1, 768, 8, 8]	1,536
│ └─ReLU: 2-16	[1, 768, 8, 8]	--
│ └─Conv2d: 2-17	[1, 768, 8, 8]	5,308,416
│ └─BatchNorm2d: 2-18	[1, 768, 8, 8]	1,536
│ └─ReLU: 2-19	[1, 768, 8, 8]	--
│ └─ConvTranspose2d: 2-20	[1, 384, 16, 16]	4,718,592

```

└─BatchNorm2d: 2-21          [1, 384, 16, 16]      768
└─ReLU: 2-22                 [1, 384, 16, 16]      --
└─ConvTranspose2d: 2-23      [1, 192, 32, 32]     1,179,648
└─BatchNorm2d: 2-24         [1, 192, 32, 32]     384
└─ReLU: 2-25                 [1, 192, 32, 32]     --
└─ConvTranspose2d: 2-26      [1, 1, 64, 64]       3,072
└─Sigmoid: 2-27             [1, 1, 64, 64]       --
=====

Total params: 20,719,616
Trainable params: 20,719,616
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 3.40
=====

Input size (MB): 0.08
Forward/backward pass size (MB): 8.95
Params size (MB): 82.88
Estimated Total Size (MB): 91.91
=====

```

```

# cfg = Config()
# cfg.bottleneck_size = 4 # 可以在這裡改 n=1, n=8

# 2. 準備資料
# 請確保字體路徑正確
# dataset = PairedFontDataset("times.ttf", "EB-Garamond.ttf", cfg)
# dataset = PairedFontDataset("YuPearl.ttf", "times.ttf", cfg)
# dataset = PairedFontDataset("YuPearl.ttf", "tegaki_zatsu.ttf", cfg)
# dataset = PairedFontDataset("GenJyuuGothic-Light.ttf", "tegaki_zatsu.ttf", cfg)
dataset = PairedFontDataset("NotoSansTC-Regular.ttf", "tegaki_zatsu.ttf", cfg)
# dataset = PairedFontDataset("kaiu.ttf", "DancingScript.ttf", cfg)

print("預覽數據集樣本...")
preview_dataset_samples(dataset, n=6)

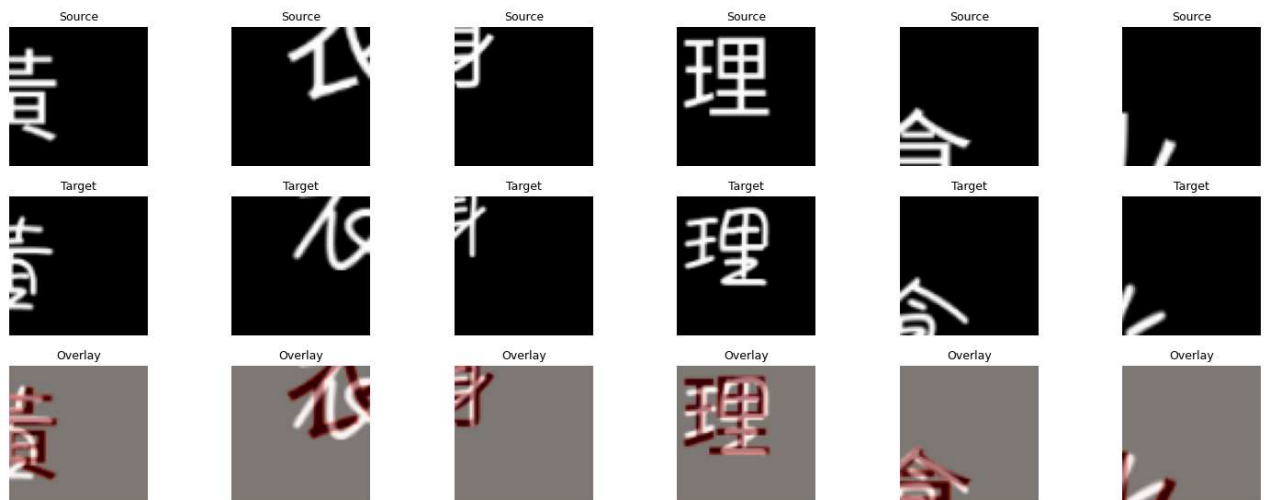
```

```

總有效字符: 292
└─ 普通訓練字符 (Common): 292 個
└─ 缺失測試字符 (Missing/Zero-shot): []
預覽數據集樣本...

```

Dataset Preview: Source (Top), Target (Mid), Overlay (Bottom)



```

from google.colab import files
print(f"--- Starting Experiment with Bottleneck Size: {cfg.bottleneck_size} ---")

dataloader = DataLoader(dataset, batch_size=cfg.batch_size, shuffle=True, num_workers=12)

G = DynamicGenerator(cfg).to(device)
D = Discriminator(cfg).to(device)

print(f"Generator parameters: {sum(p.numel() for p in G.parameters())}")
print(f"Discriminator parameters: {sum(p.numel() for p in D.parameters())}")

opt_G = optim.Adam(G.parameters(), lr=cfg.lr, betas=cfg.betas)

```

```
opt_D = optim.Adam(D.parameters(), lr=cfg.lr, betas=cfg.betas)

history = {'G_loss': [], 'D_loss': [], 'L1_loss': [], 'loss_G_adv': []}
```

```
--- Starting Experiment with Bottleneck Size: 8 ---
Generator parameters: 20719616
Discriminator parameters: 2761728
```

開始使用 AI 編寫或生成程式碼。

```
# 訓練循環
for epoch in range(cfg.num_epochs):
    for i, (src_imgs, tgt_imgs, labels) in enumerate(dataloader):
        bs = src_imgs.size(0)
        src_imgs = src_imgs.to(device)
        tgt_imgs = tgt_imgs.to(device)

        for _ in range(cfg.n_critic):
            # 1. 訓練 Discriminator
            opt_D.zero_grad()
            z = torch.randn(bs, cfg.nz, cfg.bottleneck_size, cfg.bottleneck_size).to(device)
            fake_imgs = G(z, src_imgs).detach()

            loss_D = -torch.mean(D(tgt_imgs)) + torch.mean(D(fake_imgs))
            gp = compute_gradient_penalty(D, tgt_imgs, fake_imgs, device)
            loss_D += cfg.lambda_gp * gp

            loss_D.backward()
            opt_D.step()

            # 2. 訓練 Generator (每 n_critic 次 D 訓練一次 G)
            opt_G.zero_grad()
            z = torch.randn(bs, cfg.nz, cfg.bottleneck_size, cfg.bottleneck_size).to(device)
            gen_imgs = G(z, src_imgs)

            loss_G_adv = -torch.mean(D(gen_imgs))
            loss_G_l1 = F.l1_loss(gen_imgs, tgt_imgs)

            # 這裡加入了 L1 權重參數
            loss_G = loss_G_adv + cfg.lambda_l1 * loss_G_l1

            loss_G.backward()
            opt_G.step()

            # 記錄數據
            history['G_loss'].append(loss_G.item())
            history['loss_G_adv'].append(loss_G_adv.item())
            history['D_loss'].append(loss_D.item())
            history['L1_loss'].append(loss_G_l1.item())

        # --- 每個 Epoch 結束時的動作 ---
        print(f"[Epoch {epoch}/{cfg.num_epochs}] G_loss: {loss_G.item():.4f} | L1: {loss_G_l1.item():.4f} | loss_G_adv: {loss_G_adv.item():.4f}")

    # 每 10 個 Epoch 做一次特定字符測試
    if epoch % 30 == 0 or epoch == cfg.num_epochs - 1:
        test_specific_chars(G, dataset, cfg, epoch)

    if epoch % 50 == 0 or epoch == cfg.num_epochs - 1:
        plot_training_losses(history, cfg)

if epoch == cfg.num_epochs - 1:
    pass
```